

SEARCH ENGINE RESULT RANKING CUTOFF USING C++

1. ABSTRACT

In modern computing systems, search engines are essential tools for retrieving relevant information efficiently. One critical challenge in designing such systems is determining how many results should be displayed to the user. Displaying too many results can overwhelm users and reduce clarity, while displaying too few may hide valuable information.

This project addresses the problem by applying Data Structures and Algorithms (DSA) techniques to determine the optimal number of results (k) that maximizes user engagement. The approach uses Binary Search on the answer space combined with a linear evaluation of engagement.

The solution is implemented in C++, ensuring efficiency and scalability. This case study demonstrates how algorithmic optimization improves both system performance and user experience.

2. INTRODUCTION

Search engines aim to provide users with the most relevant information based on their queries. However, deciding how many results to display is a crucial design decision.

If a system displays too many results, users may experience information overload and struggle to identify useful content. Conversely, displaying too few results may prevent users from accessing important information.

Therefore, it becomes necessary to determine an optimal cutoff point (k) that balances both aspects. This project uses DSA techniques such as Binary Search and array traversal to efficiently compute this optimal value.

3. PROBLEM STATEMENT

Given:

- A list of relevance scores sorted in descending order
- Each score represents the importance of a search result

Objective:

- Determine the optimal number of results (k)
- Maximize total engagement

Engagement Definition:

Engagement is defined as the sum of the top k relevance scores.



4. BACKGROUND & MOTIVATION

In real-world platforms such as search engines, recommendation systems, and e-commerce websites, ranking plays a significant role in user interaction.

Studies show that users tend to interact mostly with top-ranked results. Beyond a certain number of results, additional items contribute less to engagement. This creates a need for an optimized cutoff point.

This problem is a classic example of an optimization problem where we aim to find the best possible solution under given constraints.



5. KEY CONCEPTS WITH ROLE

1. Relevance Score

Definition:

A relevance score is a numerical value assigned to each search result that indicates how closely it matches the user's query. Higher values represent more important or useful results.

Why we use it:

In any search system, not all results are equally important. Relevance scores help us rank results so that the most useful ones appear first.

Role in our program:

- It acts as the **primary input data**
 - The scores are stored in a sorted array (highest to lowest)
 - They are used to calculate engagement
 - The entire decision of optimal k depends on these scores
-

2. Engagement Score

Definition:

Engagement score is defined as the total satisfaction of the user, calculated as the **sum of the top k relevance scores**.

Why we use it:

We need a measurable way to decide which value of k is best. Engagement score helps quantify how useful the displayed results are.

Role in our program:

- Used to evaluate different values of k
 - Helps compare whether increasing k improves or reduces usefulness
 - The algorithm tries to **maximize this value**
-

3. Binary Search

Definition:

Binary Search is a divide-and-conquer algorithm that repeatedly divides the search space into halves to find an optimal solution efficiently.

Why we use it:

If we check all possible values of k (from 1 to n), it would take too much time ($O(n^2)$). Binary Search reduces this to $O(n \log n)$.

Role in our program:

- Used to efficiently find the optimal value of k
 - Instead of testing all k values, it narrows down the search space
 - At each step, it checks whether increasing k improves engagement
-

4. Linear Traversal

Definition:

Linear traversal means accessing elements of an array one by one using a loop.

Why we use it:

To calculate the engagement score, we need to sum the top k values, which requires iterating through elements.

Role in our program:

- Used inside the engagement calculation function
 - Adds the first k elements of the array
 - Ensures accurate computation of engagement
-

5. Array / Vector

Definition:

An array or vector is a linear data structure used to store elements in a sequential manner.

Why we use it:

We need a structure to store and access relevance scores efficiently.

Role in our program:

- Stores the relevance scores
 - Allows quick access using index
 - Enables easy traversal and computation
-

6. Optimization Problem

Definition:

An optimization problem is one where the goal is to find the best possible solution among multiple choices.

Why we use it:

We are not just finding any value of k —we are finding the **best k that maximizes engagement**.

Role in our program:

- Defines the main objective of the algorithm
 - Guides decision-making at each step
 - Helps determine whether to increase or decrease k
-

7. Monotonic Behavior

Definition:

Monotonic behavior means a function consistently increases or decreases.

Why we use it:

Binary Search works only when there is a predictable pattern. Engagement tends to increase initially and then stabilize or decrease.

Role in our program:

- Justifies the use of Binary Search
 - Helps decide direction (left or right)
 - Ensures correctness of the approach
-



6. METHODOLOGY

The approach consists of the following steps:

1. Use Binary Search to narrow down possible values of k
2. For each selected k , calculate engagement
3. Compare engagement values
4. Adjust search space accordingly

5. Continue until optimal k is found

This method ensures efficiency compared to brute-force approaches.



7. ALGORITHM



ALGORITHM

Algorithm: Find Optimal Result Ranking Cutoff

Input: Array scores[0... $n-1$] sorted in descending order
(n = number of results)

Output: Optimal cutoff k that maximizes engagement

- 1 **Initialize:**
low $\leftarrow$ 1, high $\leftarrow$ n
bestK $\leftarrow$ 1, bestEngagement $\leftarrow$ 0
- 2 **While** low $\leq$ high:
 - mid $\leftarrow$ (low + high) / 2
 - E_{mid} $\leftarrow$ engagement of top mid elements
 - E_{next} $\leftarrow$ engagement of top (mid + 1) elements
(if mid + 1 $\leq$ n)
- 3 **If** $E_{next} > E_{mid}$:
low $\leftarrow$ mid + 1
- 4 **Else:**
bestK $\leftarrow$ mid
bestEngagement $\leftarrow$ E_{mid}
high $\leftarrow$ mid - 1
- 5 **Repeat** Steps 2 to 4 until low $>$ high.
- 6 **Return** bestK and bestEngagement.

Helper Function: engagement(k)

Returns the sum of the first k elements in scores array.

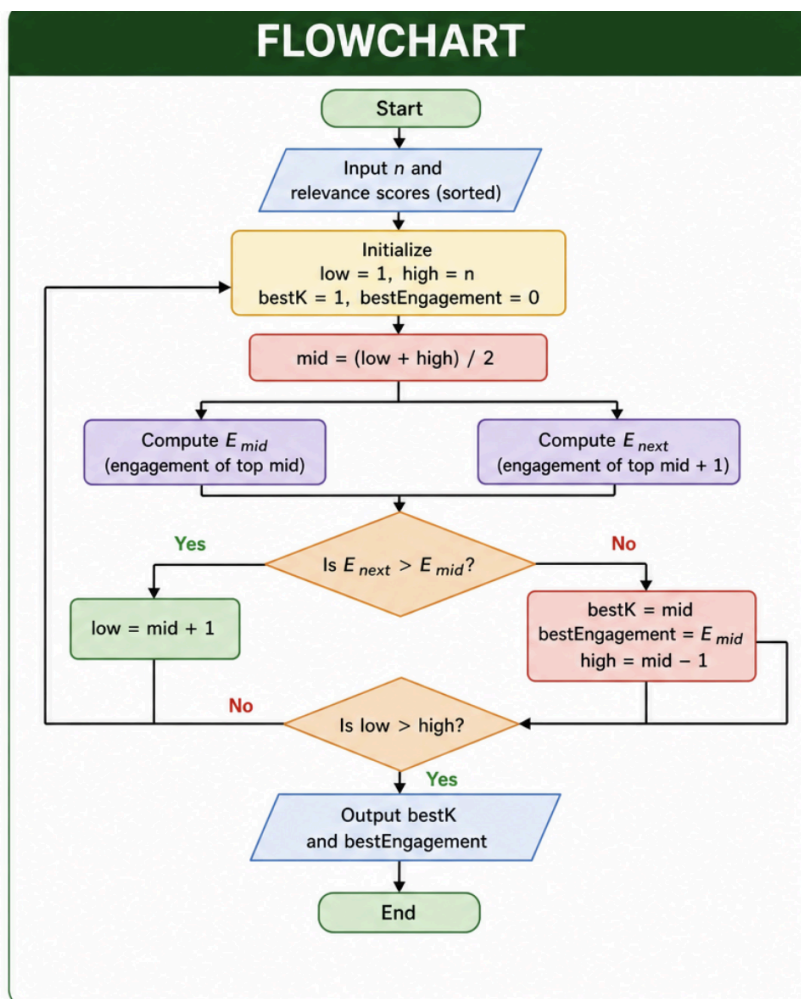
Time Complexity: $O(k)$ (worst case $O(n)$)

Step-by-Step Algorithm:

1. Initialize:
 - low = 1
 - high = n
 - bestK = 1
 - bestEngagement = 0
2. While low $\leq$ high:
 - mid = (low + high) / 2

3. Compute:
 - Engagement for mid $\rightarrow E_{mid}$
 - Engagement for mid + 1 $\rightarrow E_{next}$
4. Compare:
 - If $E_{next} > E_{mid}$
 - $\rightarrow$ Move right (low = mid + 1)
 - Else
 - $\rightarrow$ Update bestK = mid
 - $\rightarrow$ Move left (high = mid - 1)
5. Repeat until search ends
6. Return bestK and bestEngagement

8. FLOWCHART



Flow Explanation

The flowchart visually represents the working of the algorithm:

- The process starts by taking input values
- The Binary Search range is initialized
- The middle value is calculated
- Engagement values are computed
- A decision is made based on comparison
- The loop continues until optimal k is found

This visual representation helps in better understanding the algorithm logic.



9. IMPLEMENTATION DETAILS

The solution is implemented in C++ using:

- **Vectors:** To store relevance scores
- **Functions:**
 - calculateEngagement()
 - findOptimalCutoff()

Program Structure:

- Input handling
- Processing using Binary Search
- Output of results

The modular design improves readability and maintainability.



10. TIME COMPLEXITY ANALYSIS

Binary Search Complexity

Binary Search reduces the search space by half in each iteration.

Number of iterations = $\log_2(n)$

👉 Complexity = $O(\log n)$

Engagement Calculation

For each iteration:

- We sum up to k elements
- Worst case $k = n$

👉 Complexity = $O(n)$

Total Complexity

$T(n) = O(n) \times O(\log n)$

👉 $O(n \log n)$

Numerical Example

If $n = 1024$:

- Binary steps = $\log_2(1024) = 10$
- Each step takes $O(n) = 1024$

Total operations = $10 \times 1024 = 10240$

Comparison with Brute Force

Approach	Complexity
----------	------------

Brute Force	$O(n^2)$
-------------	----------

Optimized	$O(n \log n)$
-----------	---------------

11. SPACE COMPLEXITY

- Array storage = $O(n)$
- No extra data structures

👉 Total Space Complexity = $O(n)$

12. EXAMPLE & DRY RUN

Input:

[10, 8, 7, 5, 3]

Step-by-step:

k	Engagement
---	------------

1	10
---	----

2	18
---	----

3	25
---	----

4	30
---	----

5	33
---	----

Optimal k = 5

13. ADVANTAGES

- Efficient solution
 - Reduces computation time
 - Scalable for large datasets
-

14. LIMITATIONS

- Requires sorted input
 - Simplified engagement model
-

15. REAL-WORLD APPLICATIONS

- Search engines
 - E-commerce platforms
 - Recommendation systems
 - Social media feeds
-

16. FUTURE SCOPE

- Integration with machine learning
 - Personalized ranking
 - Dynamic engagement metrics
-

17. CONCLUSION

This project demonstrates how DSA techniques can be effectively applied to solve real-world optimization problems. By using Binary Search and engagement evaluation, we determine the optimal number of search results efficiently.

The approach improves both system performance and user experience, making it highly relevant for modern applications.

18. REFERENCES

1. Introduction to Algorithms – Thomas H. Cormen
2. Data Structures using C++ – Reema Thareja
3. GeeksforGeeks – Binary Search
4. TutorialsPoint – DSA Concepts
5. C++ Official Documentation

19. APPENDIX / PROJECT DETAILS

Group Members

This project was completed as a group effort by the following members:

- **Bhuvisha Gohil**
- **Prashant Yadav**
- **Himanshu Katiyara**